

Ingeniería de soluciones con AI

Semestre 2026-2

Docente:
Francisco Macaya

¿Quién soy?



Francisco Macaya

- Ingeniero Civil Industrial de la Federico Santa María
- Magister en Inteligencia Artificial en la PUC.
- Candidato a Magister en ciencia de la computación en Birkbeck, Universidad de Londres.
- Tutor en la Universidad de Montevideo para la carrera de Data Science.
- Experiencia como Líder y Data Scientist en la industria financiera y retail. (SMU, Grupo Security y Vida Camara).

Estoy afuera de Chile, así que estoy adelantado en tres horas respecto a ustedes.

*En tecnología, lo permanente
es el constante **cambio**.*

Temario del semestre

El semestre constará de las siguientes secciones:

- 1- Fundamentos de AI Generativa y Prompt Engineering.
- 2- Desarrollo de Agentes Inteligentes con AI.
- 3- Observabilidad, Seguridad y ética de Agente de AI.

Evaluación del Semestre

El semestre constará de las siguientes secciones:

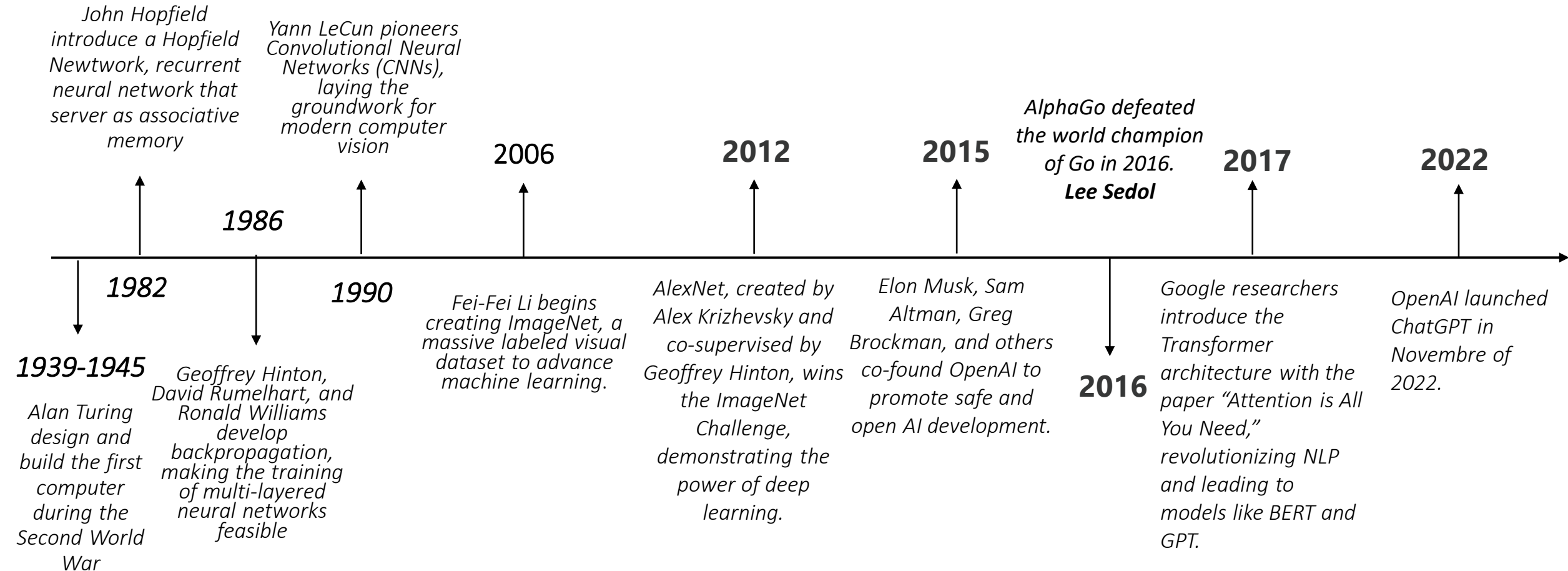
- 1- **Trabajo 1:** Fundamentos de AI Generativa y Prompt Engineering. (30%).
- 2- **Trabajo 2:** Desarrollo de Agentes Inteligentes con AI. (35%)
- 3- **Trabajo 3:** Observabilidad, Seguridad y ética de Agente de AI. (35%)
- 4- **Examen Final:** Presentación final del proyecto, evaluando todo los previos informes como bas

$$\text{Nota Final} : (0,3 * N\text{Trabajo1} + 0,35 * N\text{Trabajo2} + 0,35 * N\text{Trabajo3}) * 0,6 + 0,4 * \text{ExamenFinal}$$

Trabajo: Se debe presentar un informe, más un repositorio.

Final: Informe, Presentación en vivo, más informe.

Historia: Deep Learning



¿Qué paso aquí?



¿Qué es ChatGPT?

¿Qué es ChatGPT?

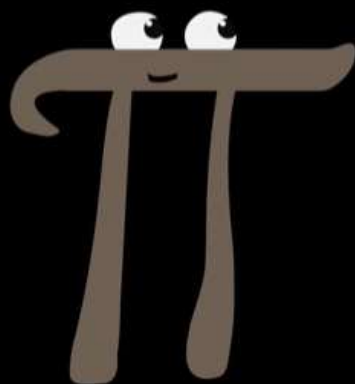
*Es un chat construido por la compañía OpenAI basado en la arquitectura GPT (**Generative Pre-*Trained* Transformer**).*



llm explained

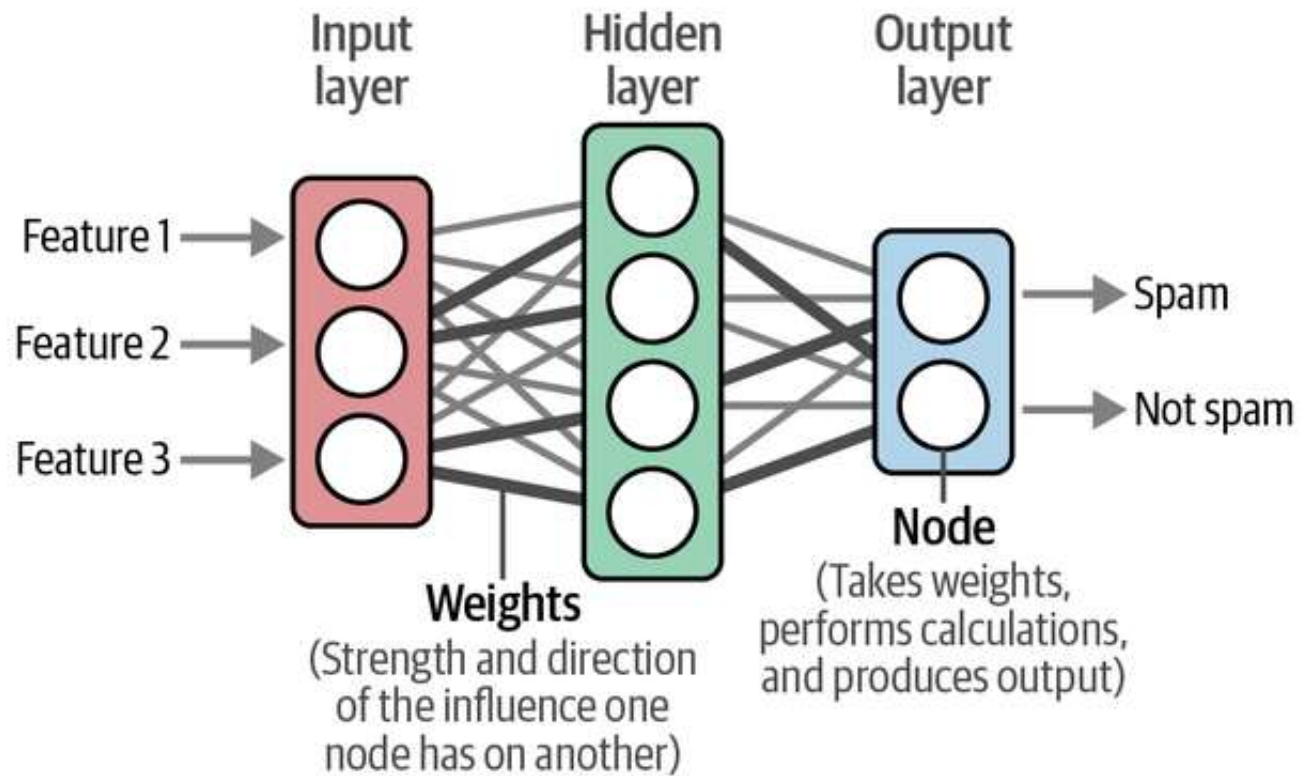


Generative Pre-trained Transformer



<https://www.youtube.com/watch?v=wjZofJX0v4M>

Deep Learning: Simulación de nuestro cerebro.



Datos:

Millones de datos para entrenamientos.

Nodos y layers:

Son capas de nodos, que tienen una función de activación, y sus entradas son multiplicadas por pesos.

Layer Final:

Capa final, que tiene una función de pérdida (Softmax), que termina calculando una probabilidad.

Se re-entrenando, actualizado los pesos hasta encontrar mínimo local.

Transformers: Attention Is All You Need

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com	Noam Shazeer* Google Brain noam@google.com	Niki Parmar* Google Research nikip@google.com	Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com	Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu	Lukasz Kaiser* Google Brain lukaszkaizer@google.com	
Illia Polosukhin* † illia.polosukhin@gmail.com			

Transformers: Attention Is All You Need

El paper plantea una nueva arquitectura llamada Transformer, la cual tiene los siguiente puntos:

- Se base en mecanismo de Attention, es decir, darle mayor peso a las palabras que son más relevantes en un frase.
- Sigue siendo autoregressive, es decir, necesita consumir cada palabra antes de crear otra nueva palabra.
- Puede ser entrenado en paralelo, lo que ayuda fuertemente a la escalabilidad en el entrenamiento.

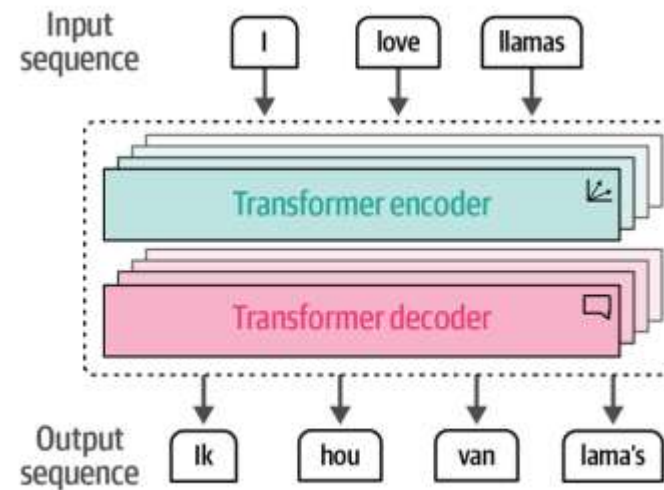


Figure 1-16. The Transformer is a combination of stacked encoder and decoder blocks where the input flows through each encoder and decoder.

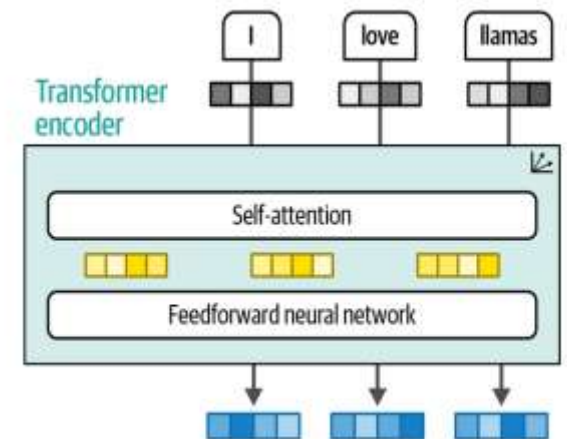


Figure 1-17. An encoder block revolves around self-attention to generate intermediate representations.

Two Layers: Encoder & Decoder

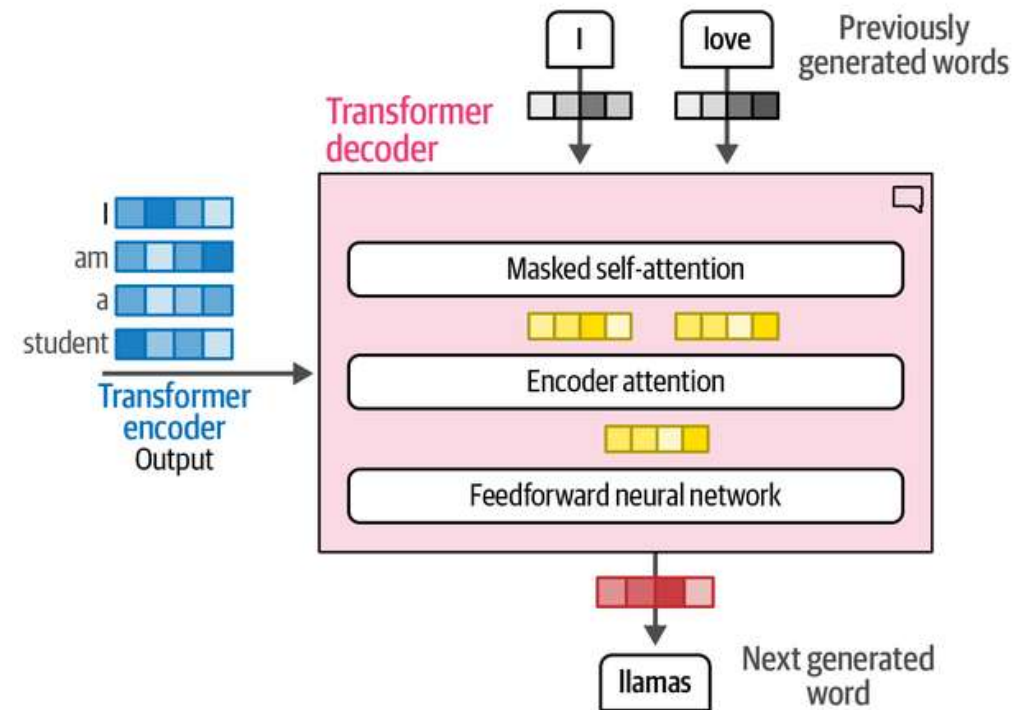
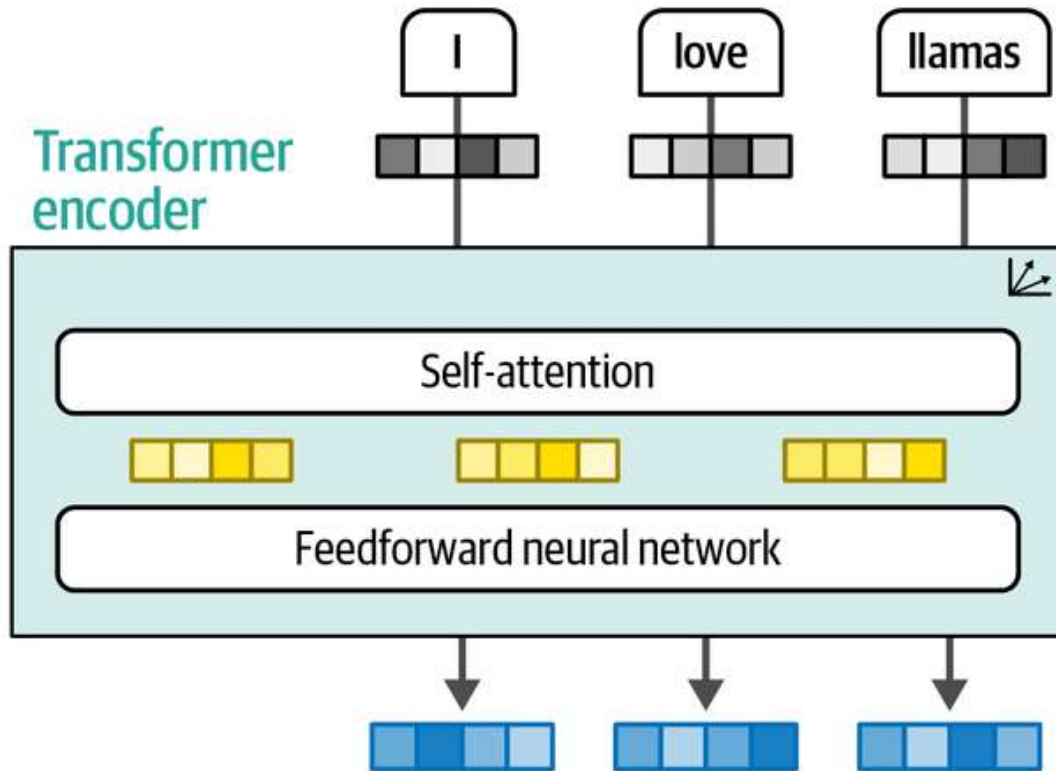
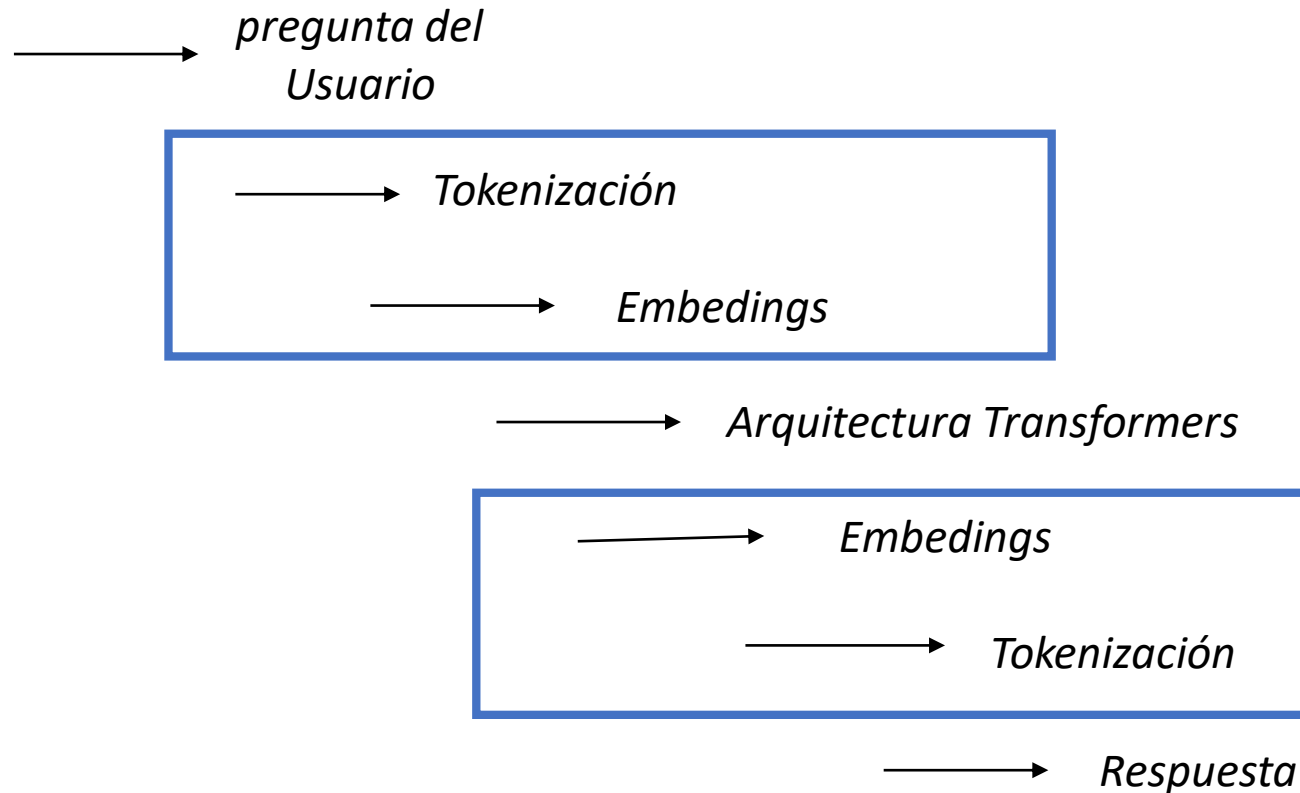


Figure 1-19. The decoder has an additional attention layer that attends to the output of the encoder.

Pipeline: ¿Cómo realmente los LLMs trabajan?



Los LLMs no reciben texto, sino que procesan números (embeddings).

Tokens: De Palabras a IDs.

Cada LLM tiene su propia tokenización, pero en general, hay cuatro tipos de técnicas para tokenizar. Estas técnicas son:

1. **Word Tokens:** Cada palabra es un token. El problema aquí es que si hay nuevas palabras, no sabremos como identificarlas.
2. **Subword Tokens:** Método contiene palabras completas y parciales. Habilidad para representar nuevas palabras por romper las demás.
3. **Character Tokens:** Cada letra es un token, y es eficiente para buscar nuevas palabras. El problema es que una palabra como play, puede significar cuatro tokens, entonces los modelos no entregan demasiado información.
4. **Byte Tokens:** Forma de representar Unicode words.

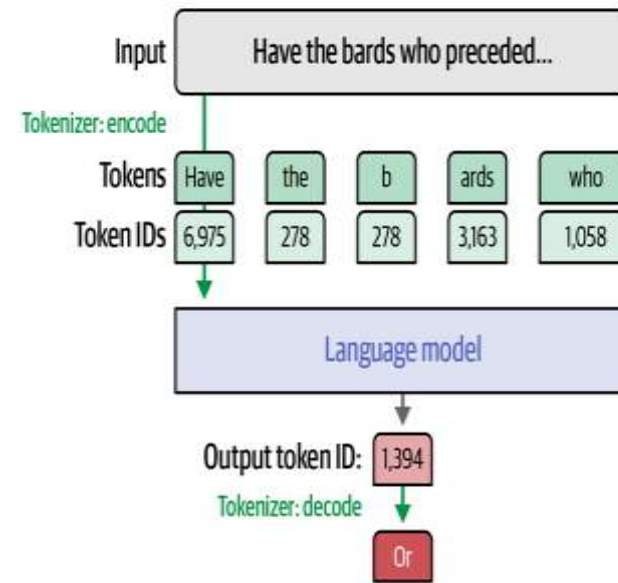


Figure 2-5. Tokenizers are also used to process the output of the model by converting the output token ID into the word or token associated with that ID.

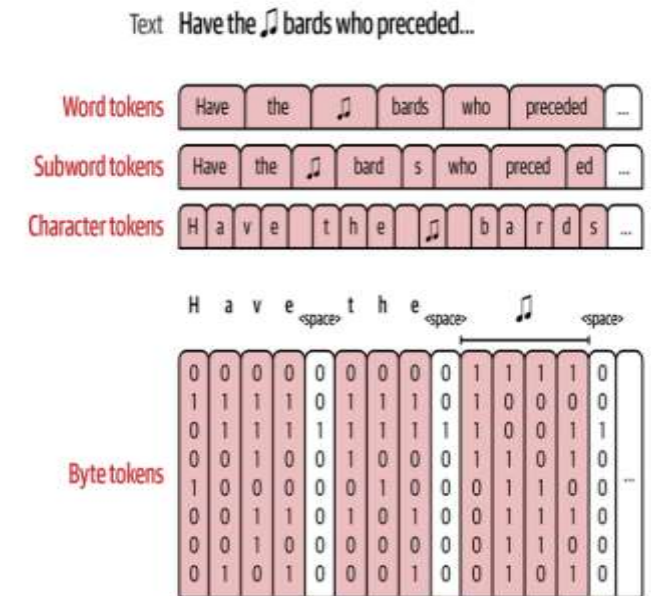


Figure 2-6. There are multiple methods of tokenization that break down the text to different sizes of components (words, subwords, characters, and bytes).

Embeddings: La representación del lenguaje en forma de números.

Token embeddings son matriz de números que representa el lenguaje, y que nos permiten construir predictores de la próxima palabras.

Como cualquier computador, los modelos no procesan palabras, sino que números, entonces es necesario antes de trabajar con los inputs, transformarlos a un idioma que sean capaz de leer el algoritmo.

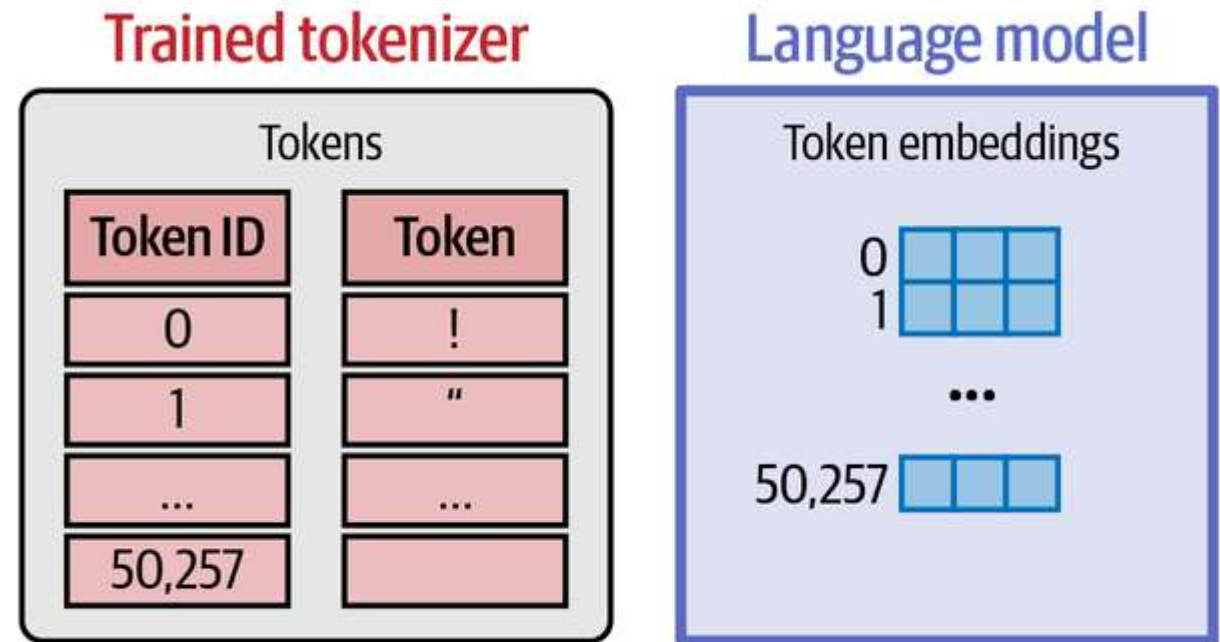
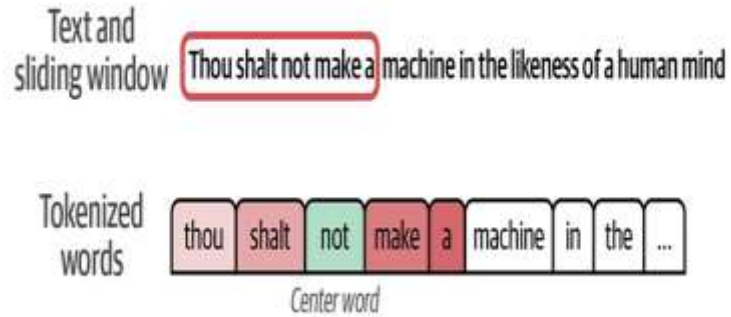


Figure 2-7. A language model holds an embedding vector associated with each token in its tokenizer.

Token Embedding: *¿Cómo crearlo?*



Paso 1: Dataset de millones de texto.

Word 1	Word 2	Target
not	thou	1
not	shalt	1
not	make	1
not	a	1
thou	apothecary	0
not	sublime	0
make	def	0
a	playback	0

Positive examples

Negative examples

Figure 2-13. We need to present our models with negative examples: words that are not usually neighbors. A better model is able to better distinguish between the positive and negative examples.

Paso 2: Construir una forma que identifique si una palabra es cercana a una frase. .

Token	Token embedding
thou	
shalt	
make	
a	
not	
apothecary	
sublime	
def	
playback	

Paso 3: Inicializar pesos aleatorios.

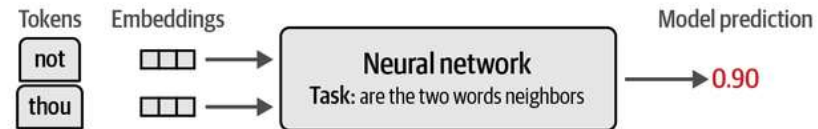


Figure 2-16. A neural network is trained to predict if two words are neighbors. It updates the embeddings in the training process to produce the final, trained embeddings.

Paso 4: Construir un modelo que prediga la variable objetivo, mediante el ajuste de los pesos de los embeddings

Attention: *The clave para los LLMs.*

Attention is a mechanism that helps the model incorporate context as it's processing a specific token. Think of the following prompt:

"The dog chased the squirrel because it"

For the model to predict what comes after "it," it needs to know what "it" refers to. Does it refer to the dog or the squirrel?

Cuando hablamos, siempre nos referimos al pasado, pero no directamente, sino que usando adjetivos, u otras formas.

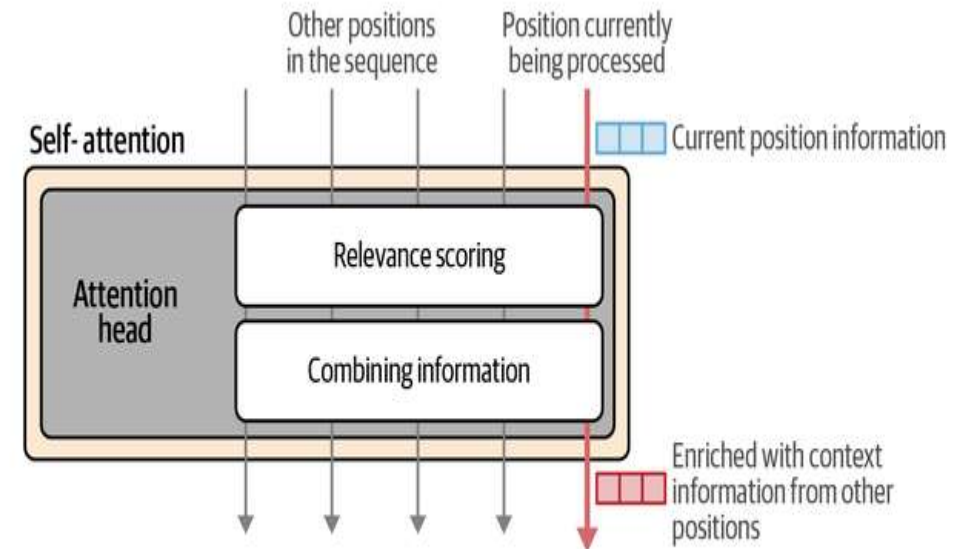


Figure 3-17. We get better LLMs by doing attention multiple times in parallel, increasing the model's capacity to attend to different types of information.

Attention: ¿Cómo se calcula?

- The attention layer (of a generative LLM) is processing attention for a single position.
- The inputs to the layer are:
 - The vector representation of the current position or token
 - The vector representations of the previous tokens
- The goal is to produce a new representation of the current position that incorporates relevant information from the previous tokens:
 - For example, if we're processing the last position in the sentence "Sarah fed the cat because it," we want "it" to represent the cat—so attention bakes in "cat information" from the cat token.
- The training process produces three projection matrices that produce the components that interact in this calculation:
 - A query projection matrix
 - A key projection matrix
 - A value projection matrix

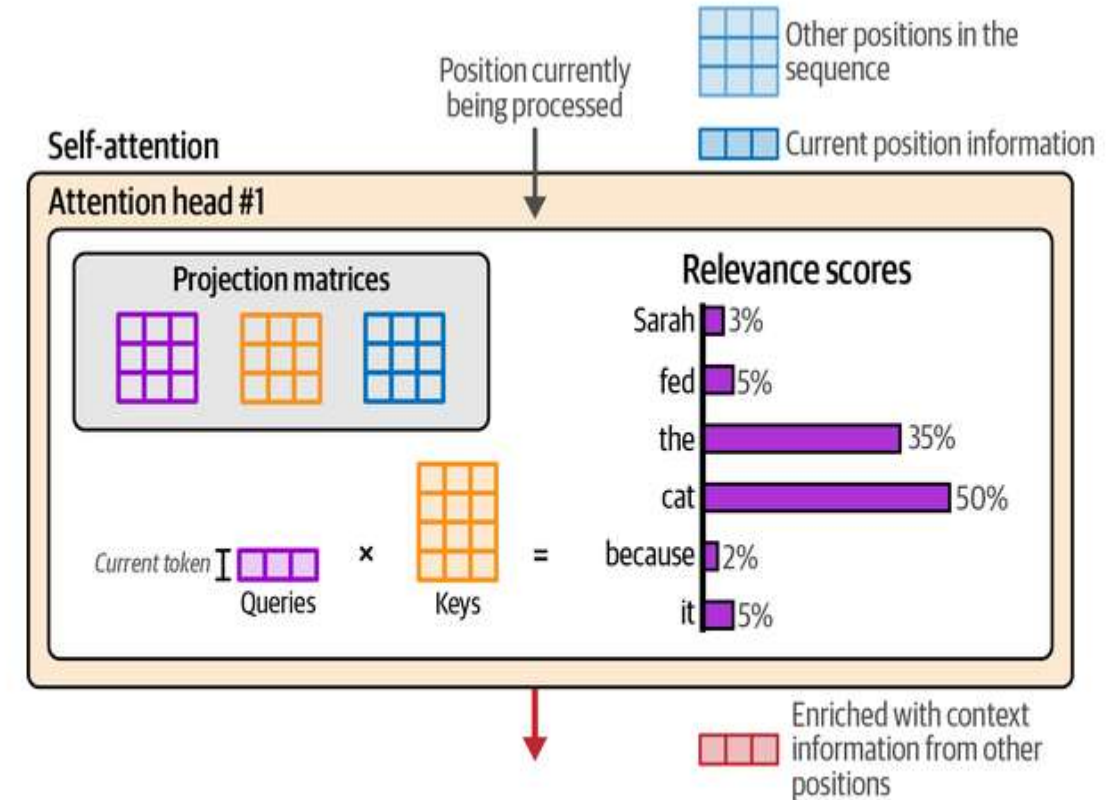


Figure 3-20. Scoring the relevance of previous tokens is accomplished by multiplying the query associated with the current position with the keys matrix.

Embeddings: *Una representación de los datos.*

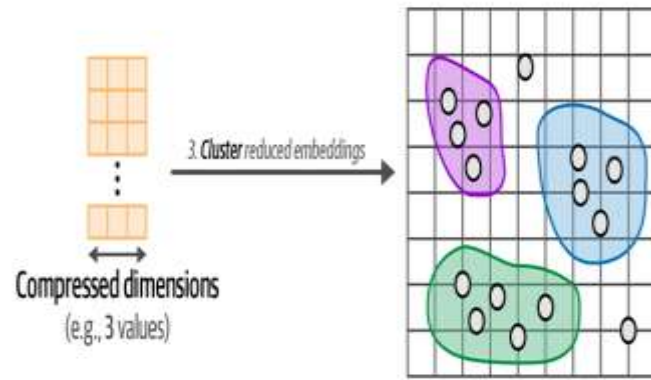
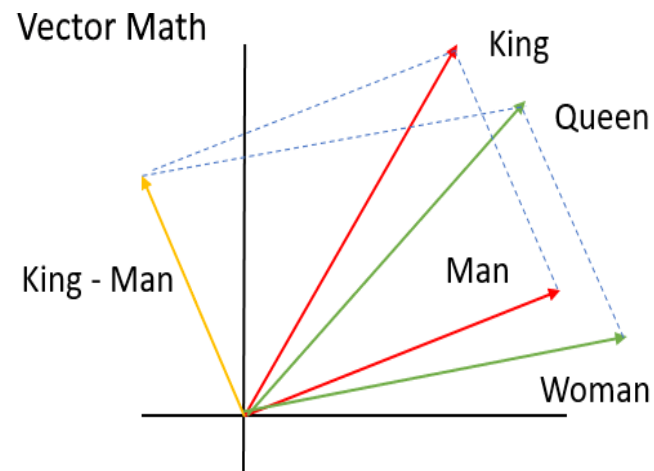


Figure 5-6. Step 3: We cluster the documents using the embeddings with reduced dimensionality.



Los embeddings son representación del lenguaje, y su semejanza con las palabras hace que podamos usar formulas matemáticas para encontrar su diferencias y similitudes.

Distancia Coseno

$$\text{Sim}(q, d_i) = \frac{\sum_{j=1}^m t_{qj} \cdot t_{ij}}{\sqrt{\sum_{j=1}^m t_{qj}^2 \cdot \sum_{j=1}^m t_{ij}^2}}$$

¿Cuáles son las empresas detrás de los foundation models?



¿Cuáles son las empresas detrás de los foundation model?

Mark Zuckerberg.
Meta



Dario Amodei
Founder/CEO Anthropic



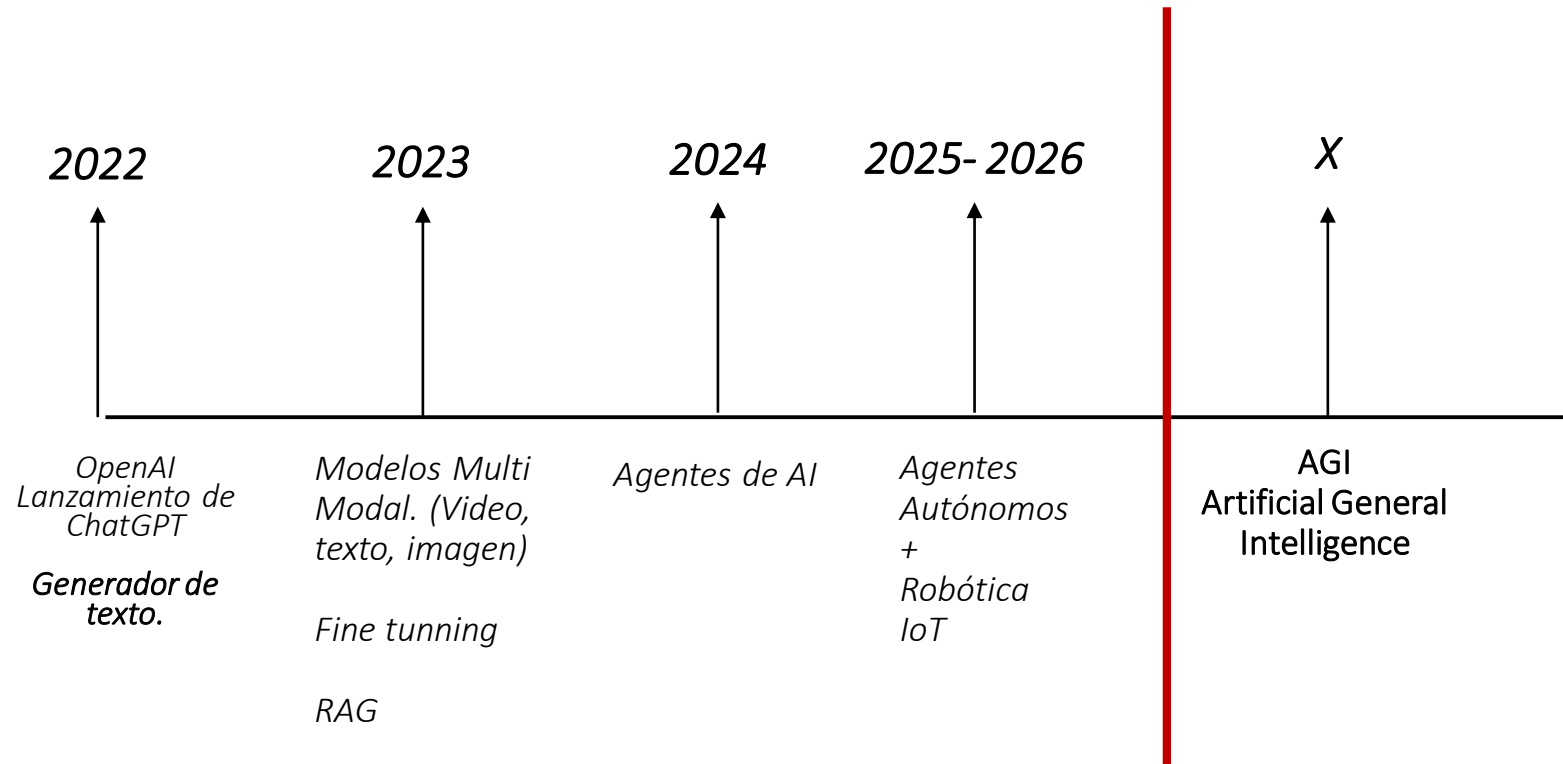
Sam Altman
OpenAI



Demis Hassabis.
Google DeepMind
AlphaFold
Premio Nobel de Química 2024

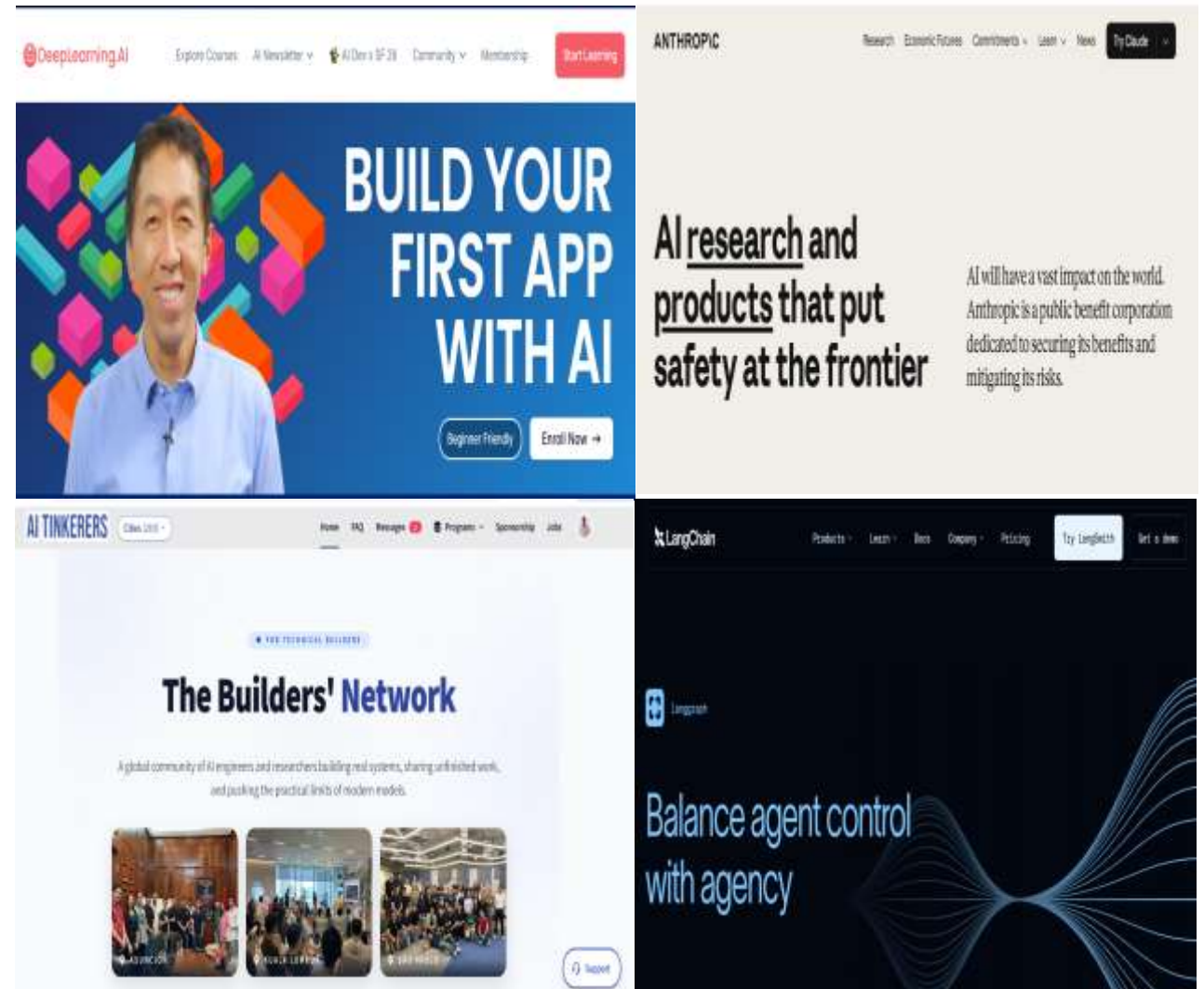


¿Cómo ha sido la evolución en los últimos tres años?

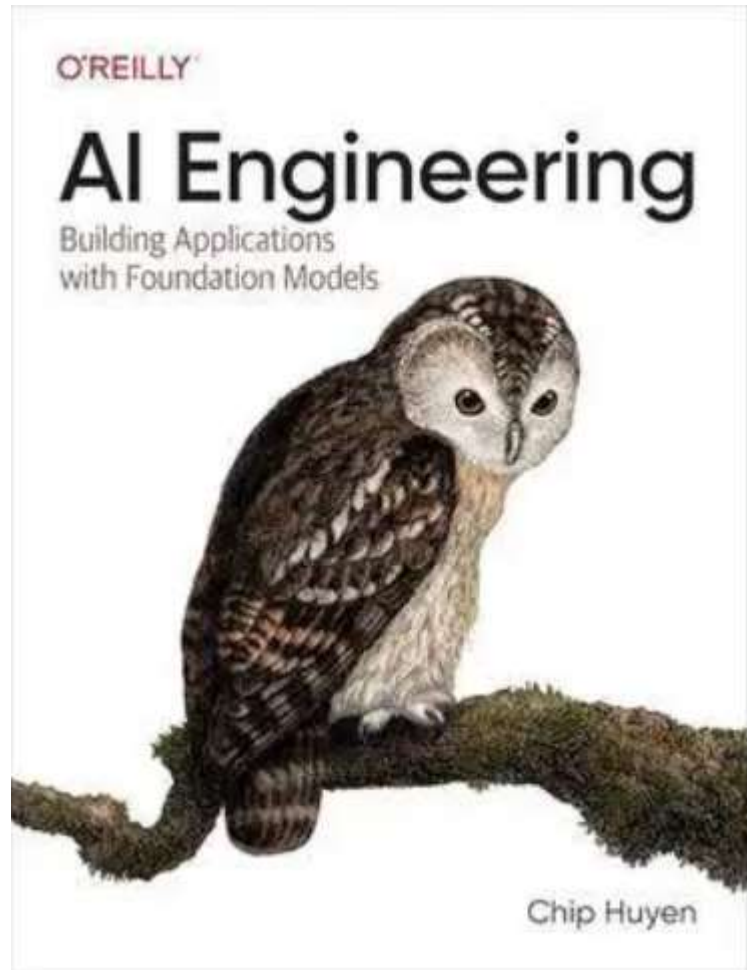


¿Dónde aprender?: Mucho ruido

- *En clases. Aprovechen de preguntar. Sino lo se, lo intentaré averiguar, y volveré con ustedes.*
- *Veán Anthropic, deeplearning, langchain, Harvard review papers, MIT, o'Reilly, papers.*
- *Júntense con personas. (compañeros, comunidades).*
- *Conózcanse, y sepan como ustedes aprenden mejor.*
- *Construyan cosas que les importen.*



Libro: AI Engineering, Chip Huyen



Lab N°1.1: Groq como proveedor de LLMs

En este laboratorio practico, se utilizará los modelos de Groq, que es una empresa que optimiza la inferencia de modelos de lenguaje opensource. Las instrucciones son las siguientes:

- 1- Acceder a esta pagina: <https://groq.com/>.*
- 2- Crear una cuenta con Google login.*
- 3- Acceder a la sección “API Keys”.*
- 4- Apretar el botón “Create API Key”.*
- 5- Crear la API KEY, y guárdala para almacenarla en el notebook.*